

Utilizando *list comprehension*

A **list comprehension** cria uma lista nova a partir de outra coleção em **uma linha**: transformar cada item, filtrar os que interessam, ou as duas coisas. É um dos recursos mais "pythônicos" da linguagem. Nesta aula: a estrutura, filtros, o if/else dentro, transformações, as versões para dicionário, set e gerador, e quando é melhor voltar para o `for` comum.

`[expr for x in seq]`

`... if condição`

`a if c else b`

`{k: v for ...}`

`{x for ...}`

`(x for ...)`

Legibilidade

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: List Comprehensions, Dictionaries, Sets; Generator expressions), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

O for + append *em uma linha*

comparar.py

```
# Com for + append
quadrados = []
for n in range(1, 6):
    quadrados.append(n ** 2)
print(quadrados)

# Com list comprehension: a mesma coisa em uma linha
quadrados = [n ** 2 for n in range(1, 6)]
print(quadrados)
```

SAÍDA

```
[1, 4, 9, 16, 25]
[1, 4, 9, 16, 25]
```

[N ** 2
o que vai na lista



FOR N IN
a variável



RANGE(1, 6)]
de onde vêm os
itens

- ✓ Lê-se: "n ao quadrado, para cada n de 1 a 5"
- ✓ Sempre cria uma **lista nova**; a original não muda
- ✓ Mais curta e, em geral, mais rápida que o for com append

Só os itens *que interessam*

filtro.py

```
notas = [8.5, 4.0, 9.5, 6.0, 7.0, 3.5]
aprovadas = [n for n in notas if n >= 7]
print(aprovadas)
nomes = ["ana", "", "bia", " ", "caio"]
validos = [n.title() for n in nomes if n.strip()]
print(validos)
```

SAÍDA

```
[8.5, 9.5, 7.0]
['Ana', 'Bia', 'Caio']
```

- ✓ O `if` no **final** filtra: só entram os itens em que a condição é verdadeira
- ✓ Dá para filtrar e transformar ao mesmo tempo (`n.title()` dos nomes válidos)
- ✓ `n.strip()` como condição: texto vazio é falso (aula 13)

Escolhendo o valor *de cada item*

if_else.py

```
notas = [8.5, 4.0, 9.5, 6.0]
status = ["aprovado" if n >= 7 else "reprovado" for n in notas]
print(status)
```

SAÍDA

```
['aprovado', 'reprovado', 'aprovado', 'reprovado']
```

POSIÇÃO DO IF

EFEITO

[x for x in seq if c]

Filtro: alguns itens ficam de fora

[a if c else b for x in seq]

Ternário (aula 13): todos entram, com valor escolhido

No começo, com `else` ; no fim, sem `else` . Trocar as posições dá `SyntaxError` .

Convertendo *dados*

transformar.py

```
precos = ["10,50", "3,99", "120,00"]
numeros = [float(p.replace(",", ".")) for p in precos]
print(numeros, sum(numeros))
palavras = "o rato roeu a roupa".split()
print([len(p) for p in palavras])
```

SAÍDA

```
[10.5, 3.99, 120.0] 134.49
[1, 4, 4, 1, 5]
```

- ✓ Uso muito comum: converter tipos de uma lista inteira
- ✓ Preços com vírgula viram `float` (aula 04) e já podem ser somados
- ✓ Qualquer expressão serve: métodos, contas, funções como `len()`

Para dicionário, set *e gerador*

dict_set.py

```
nomes = ["Ana", "Bruno", "Carla"]
tamanhos = {nome: len(nome) for nome in nomes}  # dict
print(tamanhos)
iniciais = {nome[0] for nome in ["ana", "alice", "bia"]}  # set
print(sorted(iniciais))
total = sum(n ** 2 for n in range(1, 4))  # gerador: sem []
print(total)
```

SAÍDA

```
{'Ana': 3, 'Bruno': 5, 'Carla': 5}
['a', 'b']
14
```

SINTAXE

CRIA

[expr for x in seq]

Lista

{k: v for x in seq}

Dicionário (aula 12)

{expr for x in seq}

Set, sem repetidos (aula 11)

(expr for x in seq)

Gerador: calcula um item por vez

Dentro de `sum()`, `max()` etc., o gerador dispensa os colchetes e não monta a lista inteira na memória.

Quando *não* usar

legibilidade.py

```
dados = [3, -1, 8, 0, -5, 12]
# Difícil de ler: duas condições e uma conta numa linha só
r = [x * 2 if x > 0 else 0 for x in dados if x != 0 and x > -3]
print(r)
# Mais claro com for comum
resultado = []
for x in dados:
    if x != 0 and x > -3:
        resultado.append(x * 2 if x > 0 else 0)
print(resultado)
```

SAÍDA

```
[6, 0, 16, 24]
[6, 0, 16, 24]
```

Recomendação do guia (cap. 10): comprehension para transformações **simples**, de uma linha. Com duas ou mais condições, ou lógica difícil de ler, prefira o `for` comum: **legibilidade importa**.

- ✓ As duas versões dão o mesmo resultado; a segunda se lê em voz alta
- ✓ Não use comprehension só pelos efeitos (por exemplo, para dar `print`): para isso, `for`

Explique esta list comprehension em português, passo a passo, e diga se ela ficaria mais legível como um for comum.

Limpendo *uma lista de e-mails*

Enunciado: de uma lista bagunçada, fique só com os textos que têm @, sem espaços e em minúsculas. Depois, monte o set dos domínios.

Uma resposta possível

```
# Exercício: limpar uma lista de e-mails
brutos = [" Ana@Ex.com", "bia@ex.com ", "", "CAIO@EX.COM", "x"]
emails = [e.strip().lower() for e in brutos if "@" in e]
print(emails)
dominios = {e.split("@")[1] for e in emails}
print(dominios)
```

SAÍDA

```
['ana@ex.com', 'bia@ex.com', 'caio@ex.com']
{'ex.com'}
```

- ✓ Filtro (`if "@" in e`) + transformação (`.strip().lower()`)
- ✓ Set comprehension para domínios únicos
- ✓ Desafio: conte quantos e-mails há por domínio (aula 12)

Teste o que *ficou*

1. `[x * 2 for x in [1, 2, 3]]` vale:

A) [1, 2, 3, 1, 2, 3]

B) [2, 4, 6]

C) 12

D) Erro

B.

2. `[x for x in range(6) if x % 2]` vale:

A) [0, 2, 4]

B) [1, 3, 5]

C) [0, 1, 2, 3, 4, 5]

D) []

B. Resto 1 é verdadeiro.

3. `{n: n * n for n in range(3)}` cria:

A) Uma lista

B) Um set

C) Um dicionário

D) Uma tupla

C. {0: 0, 1: 1, 2: 4}.

4. Quando preferir o for comum?

A) Nunca

B) Quando a lógica fica difícil de ler

C) Sempre

D) Só com números

B. Legibilidade importa.

Listas novas *em uma linha*

`[expressão for item in coleção if condição]` transforma e filtra numa linha só. O ternário no começo escolhe o valor de cada item; chaves criam dicionário ou set; parênteses criam um gerador. E quando a linha fica difícil de ler, o `for` comum é a melhor escolha.

`[expr for x in seq]`

`if no fim filtra`

`a if c else b`

`{k: v ...} · {x ...}`

Legibilidade

Próxima aula → 17 · Utilizando Funções

Fontes: docs.python.org/3/tutorial/datastructures.html (List Comprehensions, Sets, Dictionaries); docs.python.org/3/reference/expressions.html#generator-expressions, consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.